

1. Range GCD Query with Point Updates

Solution:

```
import java.util.*;

class Solution {
    int[] tree;

    int gcd(int a, int b) {
        while (b != 0) {
            int temp = a % b;
            a = b;
            b = temp;
        }
        return a;
    }

    void build(int[] arr, int idx, int l, int r) {
        if (l == r) {
            tree[idx] = arr[l];
            return;
        }

        int mid = (l + r) / 2;

        build(arr, 2 * idx, l, mid);
        build(arr, 2 * idx + 1, mid + 1, r);

        tree[idx] = gcd(tree[2 * idx], tree[2 * idx + 1]);
    }

    void update(int idx, int l, int r, int pos, int value) {
        if (l == r) {
            tree[idx] = value;
            return;
        }

        int mid = (l + r) / 2;

        if (pos <= mid) {
            update(2 * idx, l, mid, pos, value);
        } else {
```

```

        update(2 * idx + 1, mid + 1, r, pos, value);
    }

    tree[idx] = gcd(tree[2 * idx], tree[2 * idx + 1]);
}

int query(int idx, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) {
        return 0;
    }

    if (ql <= l && r <= qr) {
        return tree[idx];
    }

    int mid = (l + r) / 2;

    int left = query(2 * idx, l, mid, ql, qr);
    int right = query(2 * idx + 1, mid + 1, r, ql, qr);

    return gcd(left, right);
}

public void solve(int[] arr, int[][] operations) {
    int n = arr.length;
    tree = new int[4 * n];

    build(arr, 1, 0, n - 1);

    for (int[] op : operations) {
        if (op[0] == 1) {
            update(1, 0, n - 1, op[1], op[2]);
        } else {
            System.out.println(
                query(1, 0, n - 1, op[1], op[2])
            );
        }
    }
}
}

```

2. Range Product Query with Point Updates

Solution:

```

import java.util.*;

class Solution {
    static final long MOD = 1000000007L;
    long[] tree;

    void build(int[] arr, int idx, int l, int r) {
        if (l == r) {
            tree[idx] = arr[l] % MOD;
            return;
        }

        int mid = (l + r) / 2;

        build(arr, 2 * idx, l, mid);
        build(arr, 2 * idx + 1, mid + 1, r);

        tree[idx] = (tree[2 * idx] * tree[2 * idx + 1]) % MOD;
    }

    void update(int idx, int l, int r, int pos, int value) {
        if (l == r) {
            tree[idx] = value % MOD;
            return;
        }

        int mid = (l + r) / 2;

        if (pos <= mid) {
            update(2 * idx, l, mid, pos, value);
        } else {
            update(2 * idx + 1, mid + 1, r, pos, value);
        }

        tree[idx] = (tree[2 * idx] * tree[2 * idx + 1]) % MOD;
    }

    long query(int idx, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) {
            return 1;
        }

        if (ql <= l && r <= qr) {
            return tree[idx];
        }
    }
}

```

```

    }

    int mid = (l + r) / 2;

    long left = query(2 * idx, l, mid, ql, qr);
    long right = query(2 * idx + 1, mid + 1, r, ql, qr);

    return (left * right) % MOD;
}

public void solve(int[] arr, int[][] operations) {
    int n = arr.length;
    tree = new long[4 * n];

    build(arr, 1, 0, n - 1);

    for (int[] op : operations) {
        if (op[0] == 1) {
            update(1, 0, n - 1, op[1], op[2]);
        } else {
            System.out.println(
                query(1, 0, n - 1, op[1], op[2])
            );
        }
    }
}
}

```

3. Range Bitwise OR Query with Point Updates

Solution:

```

import java.util.*;

class Solution {
    int[] tree;

    void build(int[] arr, int idx, int l, int r) {
        if (l == r) {
            tree[idx] = arr[l];
            return;
        }
    }
}

```

```

int mid = (l + r) / 2;

build(arr, 2 * idx, l, mid);
build(arr, 2 * idx + 1, mid + 1, r);

tree[idx] = tree[2 * idx] | tree[2 * idx + 1];
}

void update(int idx, int l, int r, int pos, int value) {
    if (l == r) {
        tree[idx] = value;
        return;
    }

    int mid = (l + r) / 2;

    if (pos <= mid) {
        update(2 * idx, l, mid, pos, value);
    } else {
        update(2 * idx + 1, mid + 1, r, pos, value);
    }

    tree[idx] = tree[2 * idx] | tree[2 * idx + 1];
}

int query(int idx, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) {
        return 0;
    }

    if (ql <= l && r <= qr) {
        return tree[idx];
    }

    int mid = (l + r) / 2;

    return query(2 * idx, l, mid, ql, qr)
        | query(2 * idx + 1, mid + 1, r, ql, qr);
}

public void solve(int[] arr, int[][] operations) {
    int n = arr.length;
    tree = new int[4 * n];

```

```

build(arr, 1, 0, n - 1);

for (int[] op : operations) {
    if (op[0] == 1) {
        update(1, 0, n - 1, op[1], op[2]);
    } else {
        System.out.println(
            query(1, 0, n - 1, op[1], op[2])
        );
    }
}
}
}

```

4. K-th One in a Binary Array

Solution:

```

import java.util.*;

class Solution {
    int[] tree;

    void build(int[] arr, int idx, int l, int r) {
        if (l == r) {
            tree[idx] = arr[l];
            return;
        }

        int mid = (l + r) / 2;

        build(arr, 2 * idx, l, mid);
        build(arr, 2 * idx + 1, mid + 1, r);

        tree[idx] = tree[2 * idx] + tree[2 * idx + 1];
    }

    void update(int idx, int l, int r, int pos, int value) {
        if (l == r) {
            tree[idx] = value;
            return;
        }
    }
}

```

```

    }

    int mid = (l + r) / 2;

    if (pos <= mid) {
        update(2 * idx, l, mid, pos, value);
    } else {
        update(2 * idx + 1, mid + 1, r, pos, value);
    }

    tree[idx] = tree[2 * idx] + tree[2 * idx + 1];
}

int kthOne(int idx, int l, int r, int k) {
    if (l == r) {
        return l;
    }

    int mid = (l + r) / 2;

    if (tree[2 * idx] >= k) {
        return kthOne(2 * idx, l, mid, k);
    } else {
        return kthOne(
            2 * idx + 1,
            mid + 1,
            r,
            k - tree[2 * idx]
        );
    }
}

public void solve(int[] arr, int[][] operations) {
    int n = arr.length;
    tree = new int[4 * n];

    build(arr, 1, 0, n - 1);

    for (int[] op : operations) {
        if (op[0] == 1) {
            update(1, 0, n - 1, op[1], op[2]);
        } else {
            int k = op[1];

```

```

        if (tree[1] < k) {
            System.out.println(-1);
        } else {
            System.out.println(
                kthOne(1, 0, n - 1, k)
            );
        }
    }
}
}
}
}

```

5. First Element Greater Than or Equal to X in a Range

For each query $[L, R, X]$, find the smallest index i in $[L, R]$ such that $arr[i] \geq X$. Print -1 if no such index exists. The array supports point updates.

Solution:

```

import java.util.*;

class Solution {
    int[] tree;

    void build(int[] arr, int idx, int l, int r) {
        if (l == r) {
            tree[idx] = arr[l];
            return;
        }

        int mid = (l + r) / 2;

        build(arr, 2 * idx, l, mid);
        build(arr, 2 * idx + 1, mid + 1, r);

        tree[idx] = Math.max(tree[2 * idx], tree[2 * idx + 1]);
    }

    void update(int idx, int l, int r, int pos, int value) {
        if (l == r) {
            tree[idx] = value;
            return;
        }
    }
}

```



```

int mid = (l + r) / 2;

if (pos <= mid) {
    update(2 * idx, l, mid, pos, value);
} else {
    update(2 * idx + 1, mid + 1, r, pos, value);
}

tree[idx] = Math.max(tree[2 * idx], tree[2 * idx + 1]);
}

int findFirst(int idx, int l, int r, int ql, int qr, int x) {

    if (r < ql || l > qr || tree[idx] < x) {
        return -1;
    }

    if (l == r) {
        return l;
    }

    int mid = (l + r) / 2;

    int left = findFirst(
        2 * idx, l, mid, ql, qr, x
    );

    if (left != -1) {
        return left;
    }

    return findFirst(
        2 * idx + 1, mid + 1, r, ql, qr, x
    );
}

public void solve(int[] arr, int[][] operations) {
    int n = arr.length;
    tree = new int[4 * n];

    build(arr, 1, 0, n - 1);

    for (int[] op : operations) {

```

```

        if (op[0] == 1) {
            update(1, 0, n - 1, op[1], op[2]);
        } else {
            System.out.println(
                findFirst(
                    1, 0, n - 1,
                    op[1], op[2], op[3]
                )
            );
        }
    }
}

```

6. Count Zeros in a Range with Point Updates

Solution:

```

import java.util.*;

class Solution {
    int[] tree;

    void build(int[] arr, int idx, int l, int r) {
        if (l == r) {
            tree[idx] = (arr[l] == 0) ? 1 : 0;
            return;
        }

        int mid = (l + r) / 2;

        build(arr, 2 * idx, l, mid);
        build(arr, 2 * idx + 1, mid + 1, r);

        tree[idx] = tree[2 * idx] + tree[2 * idx + 1];
    }

    void update(int idx, int l, int r, int pos, int value) {
        if (l == r) {
            tree[idx] = (value == 0) ? 1 : 0;
            return;
        }

        int mid = (l + r) / 2;

```

```

        if (pos <= mid) {
            update(2 * idx, l, mid, pos, value);
        } else {
            update(2 * idx + 1, mid + 1, r, pos, value);
        }

        tree[idx] = tree[2 * idx] + tree[2 * idx + 1];
    }

    int query(int idx, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) {
            return 0;
        }

        if (ql <= l && r <= qr) {
            return tree[idx];
        }

        int mid = (l + r) / 2;

        return query(2 * idx, l, mid, ql, qr)
            + query(2 * idx + 1, mid + 1, r, ql, qr);
    }

    public void solve(int[] arr, int[][] operations) {
        int n = arr.length;
        tree = new int[4 * n];

        build(arr, 1, 0, n - 1);

        for (int[] op : operations) {
            if (op[0] == 1) {
                update(1, 0, n - 1, op[1], op[2]);
            } else {
                System.out.println(
                    query(1, 0, n - 1, op[1], op[2])
                );
            }
        }
    }
}

```

7. Longest Consecutive Ones in a Query Range

Solution:

```
import java.util.*;
```

```
class Solution {
```

```
    static class Node {
```

```
        int prefix;
```

```
        int suffix;
```

```
        int max;
```

```
        Node(int prefix, int suffix, int max) {
```

```
            this.prefix = prefix;
```

```
            this.suffix = suffix;
```

```
            this.max = max;
```

```
        }
```

```
    }
```

```
    Node[] tree;
```

```
    Node merge(Node left, Node right) {
```

```
        int prefix = left.prefix;
```

```
        int suffix = right.suffix;
```

```
        if (left.prefix == left.max && left.prefix > 0) {
```

```
            prefix = left.prefix + right.prefix;
```

```
        }
```

```
        if (right.suffix == right.max && right.suffix > 0) {
```

```
            suffix = right.suffix + left.suffix;
```

```
        }
```

```
        int max = Math.max(
```

```
            Math.max(left.max, right.max),
```

```
            left.suffix + right.prefix
```

```
        );
```

```
        return new Node(prefix, suffix, max);
```

```
    }
```

```
    void build(int[] arr, int idx, int l, int r) {
```

```
        if (l == r) {
```

```
            if (arr[l] == 1) {
```

```
                tree[idx] = new Node(1, 1, 1);
```

```

    } else {
        tree[idx] = new Node(0, 0, 0);
    }
    return;
}

int mid = (l + r) / 2;

build(arr, 2 * idx, l, mid);
build(arr, 2 * idx + 1, mid + 1, r);

tree[idx] = merge(
    tree[2 * idx],
    tree[2 * idx + 1]
);
}

void update(int idx, int l, int r, int pos, int value) {
    if (l == r) {
        if (value == 1) {
            tree[idx] = new Node(1, 1, 1);
        } else {
            tree[idx] = new Node(0, 0, 0);
        }
        return;
    }

    int mid = (l + r) / 2;

    if (pos <= mid) {
        update(2 * idx, l, mid, pos, value);
    } else {
        update(2 * idx + 1, mid + 1, r, pos, value);
    }

    tree[idx] = merge(
        tree[2 * idx],
        tree[2 * idx + 1]
    );
}

Node query(int idx, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) {

```

```

        return null;
    }

    if (ql <= l && r <= qr) {
        return tree[idx];
    }

    int mid = (l + r) / 2;

    Node left = query(2 * idx, l, mid, ql, qr);
    Node right = query(
        2 * idx + 1,
        mid + 1,
        r,
        ql,
        qr
    );

    if (left == null) return right;
    if (right == null) return left;

    return merge(left, right);
}

public void solve(int[] arr, int[][] operations) {
    int n = arr.length;
    tree = new Node[4 * n];

    build(arr, 1, 0, n - 1);

    for (int[] op : operations) {
        if (op[0] == 1) {
            update(1, 0, n - 1, op[1], op[2]);
        } else {
            Node ans = query(
                1, 0, n - 1, op[1], op[2]
            );

            System.out.println(ans.max);
        }
    }
}
}

```

8. Maximum Pair Sum in a Range

Solution:

```
import java.util.*;

class Solution {

    static class Node {
        long max1;
        long max2;

        Node(long max1, long max2) {
            this.max1 = max1;
            this.max2 = max2;
        }
    }

    Node[] tree;

    Node merge(Node a, Node b) {
        long[] values = {
            a.max1, a.max2,
            b.max1, b.max2
        };

        long max1 = Long.MIN_VALUE;
        long max2 = Long.MIN_VALUE;

        for (long x : values) {
            if (x > max1) {
                max2 = max1;
                max1 = x;
            } else if (x > max2) {
                max2 = x;
            }
        }

        return new Node(max1, max2);
    }

    void build(int[] arr, int idx, int l, int r) {
```

```

        arr[l],
        Long.MIN_VALUE
    );
    return;
}

int mid = (l + r) / 2;

build(arr, 2 * idx, l, mid);
build(arr, 2 * idx + 1, mid + 1, r);

tree[idx] = merge(
    tree[2 * idx],
    tree[2 * idx + 1]
);
}

void update(int idx, int l, int r, int pos, int value) {
    if (l == r) {
        tree[idx] = new Node(
            value,
            Long.MIN_VALUE
        );
        return;
    }

    int mid = (l + r) / 2;

    if (pos <= mid) {
        update(2 * idx, l, mid, pos, value);
    } else {
        update(2 * idx + 1, mid + 1, r, pos, value);
    }

    tree[idx] = merge(
        tree[2 * idx],
        tree[2 * idx + 1]
    );
}

Node query(int idx, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) {
        return null;
    }

```



```

    if (ql <= l && r <= qr) {
        return tree[idx];
    }

    int mid = (l + r) / 2;

    Node left = query(2 * idx, l, mid, ql, qr);
    Node right = query(
        2 * idx + 1,
        mid + 1,
        r,
        ql,
        qr
    );

    if (left == null) return right;
    if (right == null) return left;

    return merge(left, right);
}

public void solve(int[] arr, int[][] operations) {
    int n = arr.length;
    tree = new Node[4 * n];

    build(arr, 1, 0, n - 1);

    for (int[] op : operations) {
        if (op[0] == 1) {
            update(1, 0, n - 1, op[1], op[2]);
        } else {
            Node ans = query(
                1, 0, n - 1,
                op[1], op[2]
            );

            System.out.println(ans.max1 + ans.max2);
        }
    }
}
}

```

9. Range Maximum Frequency of a Value in a Sorted Array

Solution:

```
import java.util.*;
```

```
class Solution {
```

```
    static class Node {
```

```
        int leftValue;
```

```
        int rightValue;
```

```
        int leftCount;
```

```
        int rightCount;
```

```
        int maxFreq;
```

```
        Node(
```

```
            int leftValue,
```

```
            int rightValue,
```

```
            int leftCount,
```

```
            int rightCount,
```

```
            int maxFreq
```

```
        ) {
```

```
            this.leftValue = leftValue;
```

```
            this.rightValue = rightValue;
```

```
            this.leftCount = leftCount;
```

```
            this.rightCount = rightCount;
```

```
            this.maxFreq = maxFreq;
```

```
        }
```

```
    }
```

```
    Node[] tree;
```

```
    Node merge(Node a, Node b) {
```

```
        int leftValue = a.leftValue;
```

```
        int rightValue = b.rightValue;
```

```
        int leftCount = a.leftCount;
```

```
        int rightCount = b.rightCount;
```

```
        if (a.leftValue == a.rightValue &&  
            a.rightValue == b.leftValue) {
```

```
            leftCount = a.leftCount + b.leftCount;
```

```
        }
```

```

    if (b.leftValue == b.rightValue &&
        a.rightValue == b.leftValue) {

        rightCount = a.rightCount + b.rightCount;
    }

    int maxFreq = Math.max(a.maxFreq, b.maxFreq);

    if (a.rightValue == b.leftValue) {
        maxFreq = Math.max(
            maxFreq,
            a.rightCount + b.leftCount
        );
    }

    return new Node(
        leftValue,
        rightValue,
        leftCount,
        rightCount,
        maxFreq
    );
}

void build(int[] arr, int idx, int l, int r) {

    if (l == r) {
        tree[idx] = new Node(
            arr[l],
            arr[l],
            1,
            1,
            1
        );
        return;
    }

    int mid = (l + r) / 2;

    build(arr, 2 * idx, l, mid);
    build(arr, 2 * idx + 1, mid + 1, r);

    tree[idx] = merge(

```

```

        tree[2 * idx],
        tree[2 * idx + 1]
    );
}

```

```

Node query(int idx, int l, int r, int ql, int qr) {

```

```

    if (qr < l || r < ql) {
        return null;
    }

```

```

    if (ql <= l && r <= qr) {
        return tree[idx];
    }

```

```

    int mid = (l + r) / 2;

```

```

    Node left = query(2 * idx, l, mid, ql, qr);

```

```

    Node right = query(

```

```

        2 * idx + 1,
        mid + 1,
        r,
        ql,
        qr
    );

```

```

    if (left == null) return right;

```

```

    if (right == null) return left;

```

```

    return merge(left, right);
}

```

```

public void solve(int[] arr, int[][] queries) {

```

```

    int n = arr.length;

```

```

    tree = new Node[4 * n];

```

```

    build(arr, 1, 0, n - 1);

```

```

    for (int[] q : queries) {

```

```

        Node ans = query(
            1, 0, n - 1,
            q[0], q[1]
        );
    }
}

```

```

        System.out.println(ans.maxFreq);
    }
}
}

```

10. Bracket Sequence Range Query

Solution:

```

import java.util.*;

class Solution {

    static class Node {
        int open;
        int close;
        int matched;

        Node(int open, int close, int matched) {
            this.open = open;
            this.close = close;
            this.matched = matched;
        }
    }

    Node[] tree;

    Node merge(Node a, Node b) {

        int match = Math.min(a.open, b.close);

        int matched = a.matched + b.matched + match * 2;

        int open = a.open + b.open - match;
        int close = a.close + b.close - match;

        return new Node(open, close, matched);
    }

    void build(String s, int idx, int l, int r) {

        if (l == r) {

```

```

        if (s.charAt(l) == '(') {
            tree[idx] = new Node(1, 0, 0);
        } else {
            tree[idx] = new Node(0, 1, 0);
        }

        return;
    }

    int mid = (l + r) / 2;

    build(s, 2 * idx, l, mid);
    build(s, 2 * idx + 1, mid + 1, r);

    tree[idx] = merge(
        tree[2 * idx],
        tree[2 * idx + 1]
    );
}

Node query(int idx, int l, int r, int ql, int qr) {

    if (qr < l || r < ql) {
        return null;
    }

    if (ql <= l && r <= qr) {
        return tree[idx];
    }

    int mid = (l + r) / 2;

    Node left = query(
        2 * idx,
        l,
        mid,
        ql,
        qr
    );

    Node right = query(
        2 * idx + 1,
        mid + 1,
        r,

```

```

        ql,
        qr
    );

    if (left == null) return right;
    if (right == null) return left;

    return merge(left, right);
}

public void solve(String s, int[][] queries) {

    int n = s.length();
    tree = new Node[4 * n];

    build(s, 1, 0, n - 1);

    for (int[] q : queries) {

        Node ans = query(
            1,
            0,
            n - 1,
            q[0],
            q[1]
        );

        System.out.println(ans.matched);
    }
}
}

```